

Python: The Greatest Hits

*A step-by-step tutorial using real code
from the MT5HedgingEngine codebase*

12 lessons · ~30 minutes each · type the code yourself

For learners who can already write a function

How to use this guide

This guide picks twelve Python concepts that show up everywhere in real codebases — the ones that separate a script that runs from code that **scales, reads cleanly, and is easy to change later**. Each lesson has the same shape:

1. **The idea** — what the concept is, in two or three sentences.
2. **Real code from the book** — an actual snippet from the MT5HedgingEngine project, lightly trimmed.
3. **Walkthrough** — what each line is doing and why it was written that way.
4. **Try it yourself** — a small, self-contained exercise you can run in any Python REPL.

Before you start: you need Python 3.10 or newer. Open a terminal and run `python --version`. If you see something below 3.10, install a newer one before continuing — a couple of the snippets use modern syntax (like `int | None`) that older versions reject.

How to actually do this: create a folder called `greatest_hits`, and for each lesson make a file like `01_modules.py`, `02_classes.py`, and so on. **Type the code in by hand.** Don't paste it. The small typos and “wait, why does that work?” moments are where the learning lives.

About the source codebase

The examples come from **MT5HedgingEngine** — a real Python project (~51,000 lines, 72 files) that connects to the MetaTrader 5 trading platform, scrapes prop-firm dashboards in a headless browser, and serves a Flask web app on top. You don't need to understand trading to follow the tutorial — every snippet is presented standalone. The trading context is only there to give the code something real to **do**.

Where the original code contained sample credentials or URLs, I've replaced them with empty defaults or placeholders so you can read the pattern without copying anything sensitive.

Lesson 1 — Modules and packages

The idea

Every `.py` file is a **module**. A folder that contains an `__init__.py` file (even an empty one) is a **package**. When you write `from trader_companion.signals import sma`, Python finds the file `trader_companion/signals/sma.py`, runs it from top to bottom *once*, and exposes the names defined inside (functions, classes, constants) under that module. Subsequent imports get the cached version — the file is **not** re-executed.

This caching matters: if you put code with side effects (writing files, opening network connections) at module top level, that code runs the first time anything imports the module — sometimes in surprising places.

Real code from the book

The MT5HedgingEngine project organises its trading-signal calculations into a tiny package. Here is the actual layout:

```
trader_companion/  
  __init__.py  
  signals/  
    __init__.py      # 2 lines – see below  
    sma.py           # Simple Moving Average  
    ema.py           # Exponential Moving Average  
    rsi.py           # Relative Strength Index  
    macd.py          # MACD  
    bb.py            # Bollinger Bands  
    ...              # 25 more indicators
```

And here, calling code somewhere else in the project does:

```
from trader_companion.signals import sma, ema, rsi  
  
fast = sma.calculate(prices, period=10)  
slow = ema.calculate(prices, period=50)
```

Walkthrough

1. Putting an empty `__init__.py` in `signals/` tells Python “**this folder is a package, not just a folder of scripts.**”
2. Each indicator lives in its own file. That keeps each module short, focused, and easy to test in isolation.
3. Calling code uses dotted paths (`trader_companion.signals.sma`) that mirror the folder structure exactly. If you can read the import, you can find the file.
4. The first import of `sma` runs `sma.py` once. Every later import — from anywhere in the codebase — gets the same cached module object.

Try it yourself

Exercise

Create this folder structure on your machine:

```
lesson01/  
  main.py  
  mathkit/  
    __init__.py    # leave it empty  
    arithmetic.py
```

In `arithmetic.py` put:

```
print("arithmetic.py is being executed")  
  
def add(a, b):  
    return a + b  
  
def multiply(a, b):  
    return a * b
```

In `main.py` put:

```
from mathkit import arithmetic  
  
print(arithmetic.add(2, 3))  
print(arithmetic.multiply(4, 5))  
  
# Import a second time – does the print at the top of arithmetic.py run again?  
from mathkit import arithmetic  
print("done")
```

What you should see: the “`arithmetic.py` is being executed” line prints **once**, not twice. That's the import cache at work. Now delete `__init__.py` and run again — depending on your Python version you may get an `ImportError` or a deprecation warning, because that file is what marks the folder as a package.

Lesson 2 — Classes and objects

The idea

A **class** is a blueprint. Calling the class — `Trade(symbol='EURUSD', volume=0.1)` — runs the special `__init__` method to construct an **instance**. Methods take `self` as their first parameter; Python passes the instance in automatically when you call `trade.close()`. Attributes set on `self` are per-instance; attributes set at class scope are shared across all instances.

Real code from the book

Here is one of the SQLAlchemy ORM models from `dashboard/models.py`. It maps a Python class to a database table — every instance of `ApiKey` represents one row:

```
from sqlalchemy import Column, Integer, Text, SmallInteger, text
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass

class ApiKey(Base):
    __tablename__ = 'api_keys'

    id = Column(Integer, primary_key=True, autoincrement=True)
    key_hash = Column(Text, unique=True, nullable=False)
    key_prefix = Column(Text, nullable=False)
    admin = Column(Text, nullable=False)
    trader = Column(Text, nullable=False)
    scope = Column(Text, server_default=text("'full'"))
    created_at = Column(Text, nullable=False)
    is_active = Column(SmallInteger, server_default=text('1'))
```

Walkthrough

5. `class ApiKey(Base):` defines a new class that inherits from `Base` (more on inheritance in lesson 3).
6. `__tablename__` is a **class-level** attribute — every `ApiKey` instance shares the same table name. There is no `self` because the value belongs to the class, not to any one instance.
7. Each `Column(...)` line is also at class scope. SQLAlchemy reads them once when the class is defined, builds an internal description of the table, and uses that for queries.
8. When you later write `key = ApiKey(admin='alice', trader='bob', ...)`, SQLAlchemy generates an `__init__` that assigns each keyword argument to `self`. So `key.admin` is `'alice'` — that one's per-instance.

Try it yourself

Exercise

Without any database, build a tiny Trade class:

```
class Trade:
    DEFAULT_BROKER = "PlexyTrade"    # class attribute – shared

    def __init__(self, symbol, volume, side):
        self.symbol = symbol          # instance attributes – unique
        self.volume = volume
        self.side = side              # 'buy' or 'sell'
        self.is_open = True

    def close(self):
        self.is_open = False
        return f"Closed {self.side} {self.volume} {self.symbol}"

t1 = Trade("EURUSD", 0.1, "buy")
t2 = Trade("GBPUSD", 0.05, "sell")

print(t1.symbol, t2.symbol)          # different
print(t1.DEFAULT_BROKER, t2.DEFAULT_BROKER) # same
print(t1.close())
print(t1.is_open, t2.is_open)        # True/False – independent state
```

Now try this experiment: change `Trade.DEFAULT_BROKER = "NewBroker"` (note: `Trade`, not `t1`) and re-print `t1.DEFAULT_BROKER`. Both instances now see the new value, because there's still only one. That's class-attribute sharing in action.

Lesson 3 — Inheritance

The idea

A class can extend another class by listing it in parentheses: `class DevelopmentConfig(Config):`. The subclass gets every method and attribute of the parent for free, but can **override** any of them by redefining it. `super().method()` calls the parent's version — useful when you want to **extend** behaviour rather than replace it entirely.

Inheritance is one form of code reuse. It works well when subclasses really **are a** kind of the parent (a `DevelopmentConfig` **is a** `Config`). When that's not the case, prefer composition — holding another object as an attribute instead of inheriting from it.

Real code from the book

From `config/production.py` — a base config class plus three environment-specific subclasses:

```
class Config:
    """Base configuration."""
    DEBUG = False
    TESTING = False
    APP_NAME = 'MT5 Hedging Dashboard'
    APP_VERSION = '1.0.1'
    SESSION_COOKIE_SECURE = True # HTTPS only
    SESSION_COOKIE_HTTPONLY = True
    DATABASE_TYPE = os.getenv('DATABASE_TYPE', 'sqlite')

class DevelopmentConfig(Config):
    """Development overrides."""
    DEBUG = True
    SESSION_COOKIE_SECURE = False # http is fine on localhost
    DATABASE_TYPE = 'sqlite'
    LOG_LEVEL = 'DEBUG'

class ProductionConfig(Config):
    """Production overrides."""
    DEBUG = False
    SESSION_COOKIE_SECURE = True
    LOG_LEVEL = 'INFO'

class TestingConfig(Config):
    """Test overrides."""
    TESTING = True
    DATABASE_TYPE = 'sqlite' # in-memory for tests
```

Walkthrough

9. `Config` defines the defaults. Anything that's the **same** across environments lives here only once.

10. Each subclass redeclares only the values it wants to **change**.

`DevelopmentConfig.APP_NAME` doesn't appear, but the lookup walks up to the parent and returns `'MT5 Hedging Dashboard'` anyway.

- 11.** This is the classic **template + variations** pattern. Every Flask app, Django app, and most web frameworks use exactly this shape for environment configuration.
- 12.** To pick which one to use, the application reads an environment variable at startup — something like `config_class = {'dev': DevelopmentConfig, 'prod': ProductionConfig}[os.getenv('ENV')]`.

Try it yourself

Exercise

Build a tiny hierarchy. The mechanic to feel here is `super()`:

```
class PropFirm:
    def __init__(self, name, account_size):
        self.name = name
        self.account_size = account_size

    def describe(self):
        return f"{self.name} - ${self.account_size:,}"

class Topstep(PropFirm):
    def __init__(self, account_size):
        super().__init__("Topstep", account_size)  # parent's __init__
        self.daily_loss_limit = account_size * 0.03

    def describe(self):
        base = super().describe()  # parent's version
        return f"{base} (daily loss: ${self.daily_loss_limit:,.0f})"

ts = Topstep(50_000)
print(ts.describe())
# Topstep - $50,000 (daily loss: $1,500)
```

Question to sit with: what would happen if you removed the `super().__init__(...)` call from `Topstep.__init__`? Try it. The error message you get is one of the most common in Python and worth memorising.

Lesson 4 — f-strings

The idea

An **f-string** — written with an `f` prefix like `f"hello {name}"` — is Python's modern string interpolation. Anything in `{ }` is a real Python expression, evaluated and converted to a string. After a colon you can add a **format spec** that controls width, precision, alignment, and number formatting.

f-strings are compiled at parse time, so they're also the **fastest** formatting option in Python — faster than `.format()` and far faster than `%` formatting.

Real code from the book

From `config/production.py` — building a database connection URL:

```
@property
def DATABASE_URL(self):
    """Generate database URL based on configuration."""
    if self.DATABASE_TYPE == 'postgresql':
        return (
            f"postgresql://{self.POSTGRES_USER}:{self.POSTGRES_PASSWORD}"
            f"@{self.POSTGRES_HOST}:{self.POSTGRES_PORT}/{self.POSTGRES_DB}"
        )
    return f"sqlite:/// {self.SQLITE_PATH}"
```

And from `dashboard/db.py` — building a fallback path for SQLite:

```
DATABASE_URL = os.environ.get(
    'DATABASE_URL',
    f"sqlite:/// {os.path.join(os.path.dirname(os.path.abspath(__file__)), 'dashboard.db')}"
)
```

Walkthrough

13. Each `{...}` can hold any expression — attribute lookups (`self.SQLITE_PATH`), function calls (`os.path.join(...)`), arithmetic, anything.
14. Adjacent string literals concatenate at parse time, so the multi-line `postgresql://... string` is one continuous URL with no whitespace between the parts.
15. Watch the quoting: an f-string with double quotes `f"..."` can hold single quotes inside its expressions, and vice versa. Mixing them up is the most common syntax error people hit.

Try it yourself

Exercise

Try every common format spec at the REPL:

```
balance = 12345.6789
name     = "Alice"
pct      = 0.0734
n        = 7

print(f"Balance: ${balance:,.2f}")      # Balance: $12,345.68
print(f"Pct:      {pct:.1%}")           # Pct:      7.3%
print(f"Name:     {name:>10}")           # Name:          Alice (right-aligned, width 10)
print(f"Name:     {name:<10}|")          # Name:    Alice   | (left, width 10)
print(f"Hex:      {n:#06x}")            # Hex:      0x0007
print(f"Repr:     {name!r}")            # Repr:     'Alice' (calls repr())
```

The two most useful in real codebases: `: , .2f` for money, and `: .1%` for percentages.

Memorise those two and you'll cover 80% of formatting needs.

Lesson 5 — Type hints

The idea

Python is **dynamically typed at runtime** — variables don't have declared types, just whatever value is currently assigned. But you can **annotate** parameters and return values to document what the code expects:

```
def fetch(account: int) -> Trade:
    ...
```

These annotations are **not enforced** — at runtime Python ignores them. They exist for **you** (the reader), for static checkers like mypy and pyright, and for your IDE's auto-complete. The typing module supplies generics like `list[int]`, `Optional[str]`, and `dict[str, Any]`. From Python 3.10 onward, `int | None` replaces `Optional[int]` — shorter and easier to read.

Real code from the book

Type hints from across the codebase, mixed-and-matched. These are realistic shapes you'll see in production Python:

```
from typing import Any
from datetime import datetime

def get_client_profile(client_id: str) -> dict[str, Any]:
    """Fetch a client's settings from the hierarchy config."""
    ...

def parse_sheet_date(value: str | None) -> datetime | None:
    """Parse a date string from the spreadsheet, or None if missing/bad."""
    if not value:
        return None
    ...

def find_matches(
    matches: list[tuple[int, dict]],
    evaluations: list[dict],
    trade_timestamp: float | None,
) -> tuple[int, dict] | None:
    """Pick the best evaluation row for a given trade."""
    ...
```

Walkthrough

16. `client_id: str` — one annotation per parameter, separated from the type by a colon.
17. `-> dict[str, Any]` — the return type, after the parameter list. `dict[str, Any]` is “**a dict whose keys are strings and whose values can be anything.**”
18. `str | None` — the union syntax, read as “**a string or None**”. This is the modern way to express optional values, and you'll see it everywhere in code written for Python 3.10+.
19. `list[tuple[int, dict]]` — annotations nest, so “**a list of (int, dict) pairs**” reads naturally. Read these from the inside out.

Try it yourself

Exercise

Annotate this function. There's one right answer for the parameters and the return type:

```
# Before — no annotations
def biggest_winner(trades):
    if not trades:
        return None
    return max(trades, key=lambda t: t["profit"])
```

After (one good way to write it):

```
def biggest_winner(trades: list[dict]) -> dict | None:
    if not trades:
        return None
    return max(trades, key=lambda t: t["profit"])
```

Want to see hints actually catch bugs? Install mypy with `pip install mypy` and run `mypy your_file.py`. Now try calling `biggest_winner("hello")` — mypy will refuse, even though Python itself would happily run the code and crash later.

Lesson 6 — Decorators

The idea

A **decorator** is a function that wraps another function (or class) to add behaviour. The `@` syntax sitting above a function:

```
@app.route('/login')
def login():
    ...
```

is just shorthand for:

```
def login():
    ...
login = app.route('/login')(login)
```

That's it. Everything else about decorators follows from this one substitution. Decorators are how Flask attaches URL handlers, how `@property` turns a method into an attribute lookup, and how `@staticmethod` tells Python a method doesn't need `self`. When decorators stack, the one **closest to the function** runs first.

Real code from the book

From `dashboard/app.py` — a Flask route. The decorator turns a plain function into a URL handler:

```
@app.route('/maintenance')
def maintenance():
    """Show the maintenance page."""
    return render_template('maintenance.html')

@app.route('/')
def index():
    """Home page — the main dashboard."""
    if not session.get('user_id'):
        return redirect(url_for('login'))
    return render_template('index.html', user=current_user())
```

And here's a real decorator stack you'll see all over Flask code:

```
@app.route('/admin/users', methods=['POST'])
@login_required
@admin_only
def create_user():
    ...
```

Walkthrough

20. `@app.route('/maintenance')` tells Flask: “**when a request comes in for /maintenance, call this function.**” The decorator registers the function in a routing table inside the app object, then returns the function unchanged.

21. In the stacked example, decorators apply **bottom-up**. So `admin_only` wraps `create_user` first, then `login_required` wraps that wrapped version, then `app.route` registers the final wrapped thing.

22. This means `login_required` runs **before** `admin_only` when a request comes in — outer decorators run first. (The order looks reversed but read it from the request's point of view: it has to get past `login_required` before `admin_only` even gets a chance.)

Try it yourself

Exercise

Write a decorator from scratch — once you do, every `@something` you ever see will demystify:

```
import time
from functools import wraps

def timed(fn):
    @wraps(fn)                                # preserves fn.__name__, fn.__doc__
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = fn(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{fn.__name__} took {elapsed*1000:.2f} ms")
        return result
    return wrapper

@timed
def slow_add(a, b):
    """Pretend this is a real calculation."""
    time.sleep(0.05)
    return a + b

print(slow_add(2, 3))
# slow_add took 50.12 ms
# 5
```

Read this carefully: `timed` takes a function, builds a wrapper that does extra stuff around the call, and returns wrapper. The `@timed` line just rebinds `slow_add` to `timed(slow_add)`. That's the entire mechanism.

Lesson 7 — Properties

The idea

`@property` is a built-in decorator that turns a method into an attribute lookup. Instead of writing `account.equity()` (with parentheses), the caller writes `account.equity` (without). The method runs every time, but the call syntax is hidden.

Why does this matter? Because it lets you **migrate a public attribute to a computed value without breaking anyone**. You can ship `account.balance = 1000` today, and tomorrow turn `balance` into a property that reads from a database — and every caller still works unchanged.

Real code from the book

From `config/production.py` — the same database URL we saw in the f-string lesson, this time looking at the `@property` on top:

```
class Config:
    DATABASE_TYPE = 'sqlite'
    SQLITE_PATH   = 'dashboard/dashboard.db'

    @property
    def DATABASE_URL(self):
        """Generate the database URL based on configuration."""
        if self.DATABASE_TYPE == 'postgresql':
            return (
                f"postgresql://{self.POSTGRES_USER}:{self.POSTGRES_PASSWORD}"
                f"@{self.POSTGRES_HOST}:{self.POSTGRES_PORT}/{self.POSTGRES_DB}"
            )
        return f"sqlite:/// {self.SQLITE_PATH}"

config = Config()
print(config.DATABASE_URL)  # no parentheses — it computes on access
```

Walkthrough

- 23. Without `@property`, callers would have to write `config.DATABASE_URL()`. With it, `config.DATABASE_URL` alone runs the method.
- 24. Properties are read-only by default. To allow assignment, you add a `@DATABASE_URL.setter` decorator on a second method (see the exercise).
- 25. This is exactly the right tool when the **storage shape** of a value should be hidden from callers. The DB type might be SQLite or Postgres — callers shouldn't have to know, and they don't.

Try it yourself

Exercise

Build an Account class where equity is computed from balance + open positions. Then add a setter that does validation:

```
class Account:
    def __init__(self, balance):
        self._balance = balance      # leading underscore = "private by convention"
        self.open_pnl = 0.0

    @property
    def equity(self):
        return self._balance + self.open_pnl

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError(f"Balance cannot be negative: {value}")
        self._balance = value

a = Account(1000)
a.open_pnl = 250
print(a.equity)          # 1250 – computed, no parentheses

a.balance = 2000         # goes through the setter, validates
print(a.equity)          # 2250

a.balance = -100         # raises ValueError
```


Lesson 8 — Context managers (the with statement)

The idea

`with open('file.txt') as f:` guarantees that `f.close()` runs whether the block succeeds or raises an exception. The object's `__enter__` runs at the start, `__exit__` at the end. Context managers express the **acquire / release** pattern: open files, lock a mutex, start a database transaction, connect to MetaTrader 5 — and release / close / rollback / disconnect cleanly even if something goes wrong.

Real code from the book

From a data-migration script in the project — reading two JSON files into the database. Both files are opened with `with`, so they close even if the JSON is malformed:

```
import os, json

def migrate(api_keys_file, data_file):
    """Migrate data from JSON files to SQLite database."""
    migrated = {'api_keys': 0, 'clients': 0}

    if os.path.exists(api_keys_file):
        try:
            with open(api_keys_file, 'r') as f:
                old_keys = json.load(f)
                print(f"Found {len(old_keys)} API keys to migrate")
                migrated['api_keys'] = len(old_keys)
        except Exception as e:
            print(f"Error reading API keys file: {e}")

    if os.path.exists(data_file):
        try:
            with open(data_file, 'r') as f:
                data = json.load(f)
                for client_id, client_data in data.get('clients_db', {}).items():
                    save_client_data(client_id, client_data)
                    migrated['clients'] += 1
        except Exception as e:
            print(f"Error migrating client data: {e}")

    return migrated
```

Walkthrough

26. `with open(...) as f:` calls `open(...).__enter__()` which returns the file object, then binds it to `f`.

27. When the indented block ends — for any reason: success, return, an exception — `__exit__` runs, which closes the file. You don't need a `finally:` clause, the `with` gives you one for free.

28. Notice the `try / except` sits **outside** the `with`, not inside it. If `json.load` raises, the file still closes, and the error message is logged.

29. This pattern — **with-block inside try-block** — is the standard way to handle file I/O safely in Python. Memorise the shape.

Try it yourself

Exercise — write your own context manager

The fastest way to **get** context managers is to write one. Two ways:

Way 1: a class with `__enter__` and `__exit__`

```
class Timer:
    def __enter__(self):
        import time
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        elapsed = time.perf_counter() - self.start
        print(f"Took {elapsed*1000:.2f} ms")
        # return False (or None) – propagate any exception

with Timer():
    sum(i*i for i in range(1_000_000))
# Took 38.24 ms
```

Way 2: a generator with `@contextmanager` (much shorter)

```
from contextlib import contextmanager
import time

@contextmanager
def timer():
    start = time.perf_counter()
    try:
        yield # the with-block runs here
    finally:
        elapsed = time.perf_counter() - start
        print(f"Took {elapsed*1000:.2f} ms")

with timer():
    sum(i*i for i in range(1_000_000))
```

Both do the same thing. The decorator version is the bridge to the **next lesson** — generators.

Lesson 9 — Generators and yield

The idea

A function that uses `yield` instead of `return` becomes a **generator**. Calling it doesn't run the body — it returns an **iterator** that produces values one at a time, on demand. Each `yield` suspends the function; the next call resumes it right where it left off.

This is how Python streams data without holding it all in memory. `for row in read_csv(path):` can iterate a million-row file without ever loading the whole thing. `yield` from delegates to another iterator. Generators were also the historical foundation that `async / await` was later built on top of.

Real code from the book

From `dashboard/db.py` — a database session helper. This is one of the most important patterns in production Python. **Every web framework uses some version of it:**

```
from contextlib import contextmanager
from sqlalchemy.orm import sessionmaker

SessionLocal = sessionmaker(bind=engine)

@contextmanager
def get_session():
    """Dependency-style session: use as a context manager."""
    session = SessionLocal()
    try:
        yield session
        session.commit()           # success path
    except Exception:
        session.rollback()         # failure path
        raise
    finally:
        session.close()           # always runs
```

Used like this:

```
with get_session() as session:
    user = session.query(User).filter_by(email=email).first()
    user.last_login = datetime.utcnow()
    # no explicit commit – get_session does it on the way out
```

Walkthrough

30. This single function combines three concepts: a **generator** (it has `yield`), a **context manager** (the `@contextmanager` decorator turns the generator into one), and **structured exception handling** (`try/except/finally`).

31. When the caller writes `with get_session() as session:`, everything before the `yield` runs (**setup**). The yielded value becomes `session` in the `with`-block.

32. When the `with`-block ends successfully, the generator resumes after the `yield` and runs `session.commit()`. If it ended with an exception, the `except` arm runs and `session.rollback()` undoes any half-finished writes.

33. finally: runs in **both** cases — the connection always returns to the pool. This is the property that makes the pattern safe.

Try it yourself

Exercise — see lazy evaluation in action

Generators only do work when asked. Watch the print statements to see when each line actually runs:

```
def numbers(n):
    print(f"  starting up to {n}")
    for i in range(n):
        print(f"  about to yield {i}")
        yield i
        print(f"  resumed after {i}")
    print("  done")

print("creating generator...")
g = numbers(3)
print("got generator, asking for first value")
print("first:", next(g))
print("asking for second")
print("second:", next(g))
```

Notice: creating `g` prints **nothing**. The body doesn't start running until the first `next(g)`. Then it runs up to the first `yield` and pauses. The "resumed after 0" line prints on the **second** `next()`, not the first. That suspending-and-resuming is the whole trick.

Lesson 10 — Exceptions

The idea

Errors in Python **travel up the call stack** until something catches them. `try / except` catches; `raise` throws; `finally` runs cleanup either way. The most important rule: **catch the narrowest exception you can**. A bare `except:` or `except Exception:` catches everything — including `KeyboardInterrupt` on older Python and bugs you'd rather see — and hides real problems.

Real code from the book

From a sheet-parsing helper. The function **expects** some inputs to be malformed (CSVs from spreadsheets are messy), so it catches the specific failure mode and falls back gracefully:

```
from datetime import datetime

def parse_trade_date(value):
    """Parse a timestamp or ISO date string. Return None on bad input."""
    if not value:
        return None
    try:
        # First try: numeric Unix timestamp
        return datetime.fromtimestamp(float(value))
    except (ValueError, TypeError):
        # Not a number — try ISO format
        try:
            return datetime.fromisoformat(str(value).replace('Z', '+00:00'))
        except ValueError:
            return None
```

And the connection-pool initialiser, which uses a deliberately re-raising pattern:

```
def _init_pool():
    """Initialize the connection pool on first use."""
    global _connection_pool
    if _connection_pool is None:
        try:
            _connection_pool = SimpleConnectionPool(5, 15, DATABASE_URL, connect_timeout=5)
            logger.info("[DB] Connection pool initialized")
        except Exception as e:
            logger.error(f"[DB] Failed to initialize connection pool: {e}")
            raise          # log, then bubble up — don't swallow
```

Walkthrough

- 34.** `except (ValueError, TypeError):` — a tuple of specific exception classes. We know exactly which two failures we're prepared for. Anything else (e.g. `KeyboardInterrupt`) still propagates.
- 35.** The function returns `None` when input is bad, instead of crashing the caller. This is appropriate **here** because the caller can't do anything useful with a garbled date — but for unrecoverable failures, you **want** the crash.
- 36.** In the pool initialiser, the pattern is **log, then re-raise**. The catch only exists so we can write a useful error message. Without the bare `raise`, a startup failure would silently leave

`_connection_pool` as `None` and every later query would crash with a confusing message.

Try it yourself

Exercise — make a custom exception type

In real code, defining your own exception classes lets callers catch failures by **meaning**, not by accident:

```
class TradeRejected(Exception):
    """Raised when a broker refuses an order."""
    pass

class InsufficientMargin(TradeRejected):
    """More specific – not enough margin."""
    pass

def submit_order(symbol, volume, available_margin):
    needed = volume * 1000    # pretend
    if needed > available_margin:
        raise InsufficientMargin(
            f"Need ${needed:,.2f}, have ${available_margin:,.2f}"
        )
    return {"symbol": symbol, "volume": volume, "status": "filled"}

try:
    submit_order("EURUSD", 10, available_margin=500)
except InsufficientMargin as e:
    print(f"Specific catch: {e}")
except TradeRejected as e:
    print(f"General catch: {e}")
```

Why this matters: a caller that needs to handle `InsufficientMargin` specifically (e.g., to ask for more funds) can. A caller that just wants to know the order failed can catch `TradeRejected`. Inheritance lines up with the **granularity** of caller intent.

Lesson 11 — Comprehensions

The idea

`[x*2 for x in xs if x > 0]` builds a list in a single expression. There are four flavours: **list**, **set**, **dict**, and **generator**. They're usually clearer than `map/filter` and faster than the equivalent `for`-loop, because the list is sized once instead of grown one element at a time.

Generator comprehensions look the same but use (parentheses) instead of brackets. They **don't materialise** the intermediate list — `sum(x*2 for x in xs)` computes the running total without ever holding the doubled list in memory.

Real code from the book

Comprehensions are everywhere in the codebase. Here are the four flavours, drawn from typical patterns in the project:

```
# 1. List comprehension – collect open trades
open_trades = [t for t in all_trades if t.is_open]

# 2. Dict comprehension – index trades by ID
by_id = {t.id: t for t in all_trades}

# 3. Set comprehension – unique symbols traded today
symbols = {t.symbol for t in all_trades if t.opened_today}

# 4. Generator expression – sum total profit without
#     building an intermediate list
total_profit = sum(t.profit for t in all_trades if t.is_closed)
```

And one specifically pulled from the codebase — building a list of valid date matches with a filter:

```
valid_matches = [
    {
        'match': (idx, account),
        'delta': raw_delta_seconds,
        'valid_date': True,
    }
    for idx, account in matches
    if within_buffer(idx, account, BUFFER_SECONDS)
]
```

Walkthrough

37. Read every comprehension in three parts: **output expression** (left of `for`), **source** (`for x in xs`), **filter** (optional `if ...`).

38. In the multi-line example, the output is a **dictionary** being built fresh on each iteration. Comprehensions can output anything, including dicts, tuples, or other comprehensions.

39. `sum(t.profit for t in all_trades if t.is_closed)` — the parentheses-free generator expression as the **only** argument to a function. You don't need extra parens around it.

40. When you find yourself writing nested comprehensions more than two levels deep, that's the signal to switch back to a regular `for`-loop. Comprehensions are for **clarity** — when they stop

being clear, drop them.

Try it yourself

Exercise — convert each loop to a comprehension

Each of these is a real-world shape. Try rewriting each as a comprehension before peeking at the answer.

```
trades = [
    {"symbol": "EURUSD", "profit": 120.5, "closed": True},
    {"symbol": "GBPUSD", "profit": -45.0, "closed": True},
    {"symbol": "EURUSD", "profit": 30.0, "closed": False},
    {"symbol": "USDJPY", "profit": 88.2, "closed": True},
]
```

1. Symbols of all closed, profitable trades → ['EURUSD', 'USDJPY']
2. Total profit of closed trades → 163.7
3. Set of unique symbols → {'EURUSD', 'GBPUSD', 'USDJPY'}
4. Dict mapping symbol → list of its profits → {'EURUSD': [120.5, 30.0], ...}

Answers (try first, then check):

```
# 1.
[t["symbol"] for t in trades if t["closed"] and t["profit"] > 0]

# 2.
sum(t["profit"] for t in trades if t["closed"])

# 3.
{t["symbol"] for t in trades}

# 4. — this one is harder, easier as a regular loop:
from collections import defaultdict
by_symbol = defaultdict(list)
for t in trades:
    by_symbol[t["symbol"]].append(t["profit"])
# (You CAN do it as a dict comprehension with a nested list comp,
# but a defaultdict loop is far more readable. Knowing when NOT to
# use a comprehension is part of the skill.)
```


Lesson 12 — Dataclasses

The idea

A `@dataclass` auto-generates `__init__`, `__repr__`, and `__eq__` from a class's annotated fields. It removes the boilerplate from **plain data** objects. There are two rules to remember:

- Use `field(default_factory=list)` for mutable defaults. **Never** write `= []` at class scope — that one list gets shared across every instance and you'll spend a long afternoon figuring out why.
- `frozen=True` makes instances immutable and hashable — they can be put in sets and used as dict keys.

Real code from the book

The `MT5HedgingEngine` project leans heavily on SQLAlchemy ORM models for its persistent objects, so it doesn't use `@dataclass` for those. But the same shape comes up constantly for **transient** objects. Here's what a `Trade` dataclass would look like, modeled on the trade dictionaries the project actually passes around:

```
from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class Trade:
    symbol: str
    volume: float
    side: str # 'buy' or 'sell'
    open_price: float
    open_time: datetime
    comment: str = "" # default value
    tags: list[str] = field(default_factory=list) # mutable default
    is_open: bool = True

t = Trade("EURUSD", 0.10, "buy", 1.0834, datetime.utcnow())
print(t)
# Trade(symbol='EURUSD', volume=0.1, side='buy', open_price=1.0834,
#       open_time=..., comment='', tags=[], is_open=True)

# Equality is by field value, automatically:
t2 = Trade("EURUSD", 0.10, "buy", 1.0834, t.open_time)
print(t == t2) # True
```

Walkthrough

41. The `@dataclass` decorator inspects the class body, finds every field with a type annotation, and writes `__init__(self, symbol, volume, side, ...)` for you. Saving 5–10 lines per class adds up.
42. `__repr__` is also generated, so `print(t)` produces the readable output above instead of `<Trade object at 0x...>`. This alone is worth the decorator.

43. `field(default_factory=list)` calls `list()` **once per instance** to produce a fresh empty list. Compare to the bug below.

Try it yourself

Exercise — see the mutable-default trap

Run this. The output is one of Python's classic gotchas, and `@dataclass` actively protects you from it:

```
# THE BUG (works for one-off classes, breaks for dataclasses)
class Bag:
    def __init__(self, items=[]):    # ← shared default!
        self.items = items

a = Bag()
a.items.append("apple")
b = Bag()
print(b.items)                    # ['apple'] – wat

# THE FIX (regular class)
class Bag:
    def __init__(self, items=None):
        self.items = items if items is not None else []

# THE DATACLASS WAY (concise, can't get it wrong)
from dataclasses import dataclass, field

@dataclass
class Bag:
    items: list = field(default_factory=list)

a = Bag(); a.items.append("apple")
b = Bag()
print(b.items)                    # [] – independent
```

Bonus: add `frozen=True` to the `@dataclass` decorator and try assigning to `a.items`. You'll get a `FrozenInstanceError` — the instance is now immutable and can be used in a `set()` or as a dict key.

Where to go next

You've now seen the twelve concepts that show up most heavily in real Python codebases. To make the knowledge stick, **build something**. A small CLI tool, a script that scrapes one of your own data sources, a Flask app that does anything at all. The patterns in this guide will start to feel like instinct only when you reach for them yourself.

Concepts I deliberately skipped

The original book also covers these — they're worth circling back to once the twelve above are comfortable:

- **Async / await** — coroutines for I/O-bound concurrency. Most useful when you're making lots of HTTP requests or websocket reads.
- **Lambdas** — single-expression anonymous functions. Useful with `sorted(..., key=lambda x: x.field)`, and not much else. If your lambda needs more than one expression, write a `def` instead.
- **Dunder methods** beyond `__init__` — `__repr__`, `__eq__`, `__len__`, `__iter__`, `__getitem__`. Defining these lets your classes integrate with built-in syntax (`len(obj)`, `for x in obj:`, `obj[key]`).
- **@classmethod and @staticmethod** — alternate constructors and namespace-only functions inside a class.
- **Module-level state** — the global-ish caches and singletons. Powerful, easy to misuse, makes tests harder. Reach for it sparingly.

How to keep practising

- 44. Read code that's better than yours.** The MT5HedgingEngine reference book you started with is one good source. So is any well-maintained library on PyPI — pick something you use (`requests`, `flask`, `rich`) and click through to its source.
- 45. Add type hints to a script you've already written.** Then run `mypy` against it. The errors `mypy` raises will teach you things about your own code you didn't know.
- 46. Refactor a function into a class. Then refactor a class into a dataclass.** Feeling the difference is how the rationale for each becomes intuition.
- 47. Build the smallest possible web app.** Five lines of Flask, one route, one template. Then add a second route with a decorator stack. Then make the route depend on a database session via a `@contextmanager` generator. By the end you'll have used six of the twelve lessons in one project.

Good luck. The fact that you're reading something this dense at all is the main predictor of getting good — keep going.